

WonderFullyHE: математическое описание API

Назначение библиотеки

WonderFullyHE реализует библиотечный слой для защищённых вычислений над вещественными данными на базе схемы CKKS из Microsoft SEAL. Публичный API скрывает низкоуровневые типы SEAL и предоставляет:

- настройку CKKS-профилей;
- кодирование, шифрование, вычисления, расшифрование и декодирование;
- контроль численной ошибки;
- ABFT-проверки корректности вычислений;
- rotation-based линейные преобразования;
- вычисление полиномов от ciphertext;
- диагностику вычислительной глубины и bootstrapping-конвейер.

CKKS-профиль

Профиль CKKS задаётся набором

$$P = (N, Q, \Delta, S),$$

где:

- N — степень полиномиального модуля;
- $Q = (q_0, q_1, \dots, q_l)$ — цепочка коэффициентных модулей, в коде задаётся битовыми размерами;
- Δ — масштаб CKKS;
- S — число используемых слотов, $S \leq N/2$.

Публичный модуль `m2424::profiles` задаёт готовые профили, чтобы пользователь не собирал параметры CKKS вручную:

Профиль	N	S	Q , бит	Δ	Оценка глубины
<code>fast_demo_ckks</code>	4096	2048	40 – 30 – 30	2^{30}	1
<code>basic_ckks</code>	8192	4096	60 – 40 – 40 – 60	2^{40}	2
<code>balanced_ckks</code>	8192	4096	50 – 40 – 40 – 40 – 48	2^{40}	3
<code>depth_ckks</code>	16384	8192	60 – 40 – 40 – 40 – 40 – 60	2^{40}	4
<code>high_precision_ckks</code>	16384	8192	60 – 50 – 50 – 50 – 60	2^{50}	3

`fast_demo_ckks` нужен для быстрых локальных запусков. `basic_ckks` используется в основных демонстрациях. `balanced_ckks` даёт дополнительный уровень умножения при той же степени $N = 8192$. `depth_ckks` используется для анализа глубины и bootstrapping-блоков. `high_precision_ckks` увеличивает масштаб до 2^{50} , когда приоритетом является меньшая ошибка округления, а не скорость.

Готовые профили являются пресетами. Целевой способ выбора параметров — `CkksParameterPlanner`

$$(\varepsilon, D, S, \text{ops}, \text{security}) \mapsto (N, Q, \Delta),$$

где ε — целевая ошибка, D — требуемая мультипликативная глубина, S — число слотов.

Для целевой ошибки используется оценка:

$$b_{\text{result}} = \lceil -\log_2(\varepsilon) \rceil,$$

$$b_{\text{work}} = b_{\text{result}} + b_{\text{loss}}(\text{ops}),$$

где b_{loss} задаётся калибровкой конкретного набора операций. Для $\varepsilon = 10^{-9}$ имеем

$$b_{\text{result}} = 30.$$

Текущий benchmark для двух последовательных `mul_relin_rescale` даёт потерю порядка 14 бит, поэтому практический быстрый режим выбирает рабочие модули около 45 бит и $\log_2(\Delta) \approx 45$.

Адаптер над Microsoft SEAL

Создание контекста:

$$\text{SealAdapter}::\text{create}(P) \mapsto \text{SEALContext}(P).$$

Генерация ключей:

$$\text{keygen}(r, g) \mapsto (\text{sk}, \text{pk}, \text{rlk}^{(r)}, \text{gk}^{(g)}),$$

где $r, g \in \{0, 1\}$ задают необходимость Relin- и Galois-ключей.

Для оптимизации размера Galois-ключей поддерживается генерация только заданного набора ротаций:

$$\text{keygen}(K, r) \mapsto (\text{sk}, \text{pk}, \text{rlk}^{(r)}, \{\text{gk}_k\}_{k \in K}),$$

где K — множество rotation steps.

Сериализация объектов:

$$\begin{aligned} \text{save_public_key}() &\mapsto B_{\text{pk}}, & \text{save_secret_key}() &\mapsto B_{\text{sk}}, \\ \text{save_relin_keys}() &\mapsto B_{\text{rlk}}, & \text{save_galois_keys}() &\mapsto B_{\text{gk}}, & \text{save_cipher}(c) &\mapsto B_c. \end{aligned}$$

Обратные операции `load_*` восстанавливают объект в новом CKKS-контексте с тем же профилем. Это позволяет разделять контексты: контекст с секретным ключом выполняет расшифрование, а вычислительный контекст получает только ciphertext и evaluation keys.

Кодирование и шифрование:

$$\begin{aligned} \text{encode}(\mathbf{x}) &= \text{Encode}_{\Delta}(\mathbf{x}) = p, \\ \text{encrypt}(p) &= \text{Enc}_{\text{pk}}(p) = c. \end{aligned}$$

Расшифрование и декодирование:

$$\begin{aligned} \text{decrypt}(c) &= \text{Dec}_{\text{sk}}(c) = p, \\ \text{decode}(p) &= \text{Decode}_{\Delta}(p) = \hat{\mathbf{x}}. \end{aligned}$$

Для операций ciphertext–plaintext используется кодирование на уровне заданного ciphertext:

$$\text{encode_like}(\mathbf{a}, c) = \text{Encode}_{\text{scale}(c), \text{level}(c)}(\mathbf{a}).$$

Гомоморфные операции

Сложение:

$$\text{add}(c_1, c_2) = c_1 + c_2.$$

Вычитание:

$$\text{sub}(c_1, c_2) = c_1 - c_2.$$

Сложение и вычитание с plaintext:

$$\text{add_plain}(c, p) = c + p, \quad \text{sub_plain}(c, p) = c - p.$$

Умножение ciphertext на plaintext с rescale:

$$\text{mul_plain_rescale}(c, p) = \text{Rescale}(c \cdot p).$$

Умножение с релинеаризацией и rescale:

$$\text{mul_relin_rescale}(c_1, c_2) = \text{Rescale}(\text{Relin}_{\text{rlk}}(c_1 \cdot c_2)).$$

При умножении масштаб растёт:

$$\Delta \cdot \Delta = 2^{40} \cdot 2^{40} = 2^{80}.$$

Операция `Rescale` делит результат на следующий примерно 40-битный модуль и возвращает масштаб к значению, близкому к 2^{40} . При этом расходуется один уровень modulus chain.

Ротация CKKS-слов:

$$\text{rotate}(c, k) = \text{Rot}_k(c).$$

Для сложения результатов разных степеней поддерживается выравнивание уровня и масштаба:

$$\text{match_level_and_scale}(c, t) \mapsto c^* : \text{level}(c^*) = \text{level}(t), \quad \text{scale}(c^*) = \text{scale}(t).$$

Линейные преобразования через ротации

Модуль `LinearTransform` применяет преобразование:

$$L(c) = \sum_{i=0}^{m-1} a_i \cdot \text{Rot}_{k_i}(c),$$

где a_i — plaintext-коэффициенты, а k_i — шаги ротации.

В коде каждый член вычисляется как:

$$t_i = \text{mul_plain_rescale}(\text{rotate}(c, k_i), \text{encode_like}(a_i, \text{rotate}(c, k_i))),$$

после чего члены складываются на одном уровне.

Функция `sum_slots` считает сумму первых m слотов:

$$s = \sum_{i=0}^{m-1} x_i.$$

Для $m = 2^d$ используется последовательность ротаций:

$$1, 2, 4, \dots, 2^{d-1}.$$

Результат суммы помещается в первый CKKS-слот.

Вычисление полиномов

Модуль `PolynomialEvaluator` вычисляет полином:

$$P(c) = \sum_{j \in D} \alpha_j c^j,$$

где D — набор используемых степеней, а α_j — вещественные коэффициенты.

Степени ciphertext строятся последовательными умножениями:

$$c^{j+1} = \text{mul_relin_rescale}(c^j, \text{match_level_and_scale}(c, c^j)).$$

Каждый член полинома умножается на plaintext-коэффициент:

$$\alpha_j c^j = \text{mul_plain_rescale}(c^j, \text{encode_scalar_like}(\alpha_j, c^j)).$$

Перед сложением членов выполняется выравнивание уровня и масштаба. Этот блок используется как программная основа для этапа `EvalMod`.

Диагностика ciphertext

Для ciphertext c библиотека возвращает набор параметров:

$$\text{info}(c) = (\text{scale}(c), \text{chain_index}(c), \text{coeff_modulus_size}(c), \text{ciphertext_size}(c)).$$

Также измеряется сериализованный размер:

$$\text{serialized_size}(c) = |\text{Serialize}(c)|.$$

Эти значения используются в benchmark, анализе глубины и bootstrapping-отчёте.

Контроль точности

Пусть $\mathbf{y}$ — эталонный результат CPU-вычисления, а $\hat{\mathbf{y}}$ — результат после гомоморфного вычисления, расшифрования и декодирования. Для векторов одинаковой длины n :

$$\text{max_abs_error} = \max_{0 \leq i < n} |y_i - \hat{y}_i|,$$

$$\text{mean_abs_error} = \frac{1}{n} \sum_{i=0}^{n-1} |y_i - \hat{y}_i|.$$

Критерий прохождения проверки:

$$\text{ok} = \text{max_abs_error} \leq \text{tolerance}.$$

Для доказуемого контракта точности ошибка должна распространяться по стадиям:

$$E_{\text{total}} \leq E_{\text{encode}} + E_{\text{encrypt}} + E_{\text{rescale}} + E_{\text{keyswitch}} + E_{\text{poly}} + E_{\text{linear}}.$$

Планировщик должен запускать вычисление только если рассчитанная верхняя граница не превышает заданную **tolerance**.

ABFT-проверки

Контрольная сумма полезной нагрузки:

$$\text{checksum}(\mathbf{x}) = \sum_{i=0}^{m-1} x_i.$$

Добавление checksum-слота:

$$\mathbf{x}' = (x_0, x_1, \dots, x_{m-1}, \text{checksum}(\mathbf{x})).$$

Проверка добавленного checksum-слота:

$$\text{expected} = \sum_{i=0}^{m-1} y_i, \quad \text{observed} = y_m,$$

$$\text{abs_error} = |\text{expected} - \text{observed}|, \quad \text{ok} = \text{abs_error} \leq \text{tolerance}.$$

Для сложения и вычитания используются линейные свойства:

$$\text{checksum}(\mathbf{x} + \mathbf{y}) = \text{checksum}(\mathbf{x}) + \text{checksum}(\mathbf{y}),$$

$$\text{checksum}(\mathbf{x} - \mathbf{y}) = \text{checksum}(\mathbf{x}) - \text{checksum}(\mathbf{y}).$$

Для ротации используется инвариант суммы:

$$\text{checksum}(\text{Rot}_k(\mathbf{x})) = \text{checksum}(\mathbf{x}).$$

Для умножения текущая проверка сравнивает сумму результата с CPU-эталоном по-компонентного произведения:

$$\text{expected} = \sum_{i=0}^{m-1} x_i y_i, \quad \text{observed} = \sum_{i=0}^{m-1} \widehat{(xy)}_i.$$

Анализ вычислительной глубины

В профиле `depth_ckks` библиотека анализирует цепочку последовательных зашифрованных возведений в квадрат:

$$x \rightarrow x^2 \rightarrow x^4 \rightarrow x^8 \rightarrow x^{16}.$$

Каждый шаг вызывает:

$$c_{i+1} = \text{mul_relin_rescale}(c_i, c_i).$$

После каждого успешного шага фиксируются:

$$\text{scale}(c_i), \quad \text{chain_index}(c_i), \quad \text{serialized_size}(c_i), \quad \text{error}(c_i).$$

На текущем профиле следующий переход к x^{32} останавливается, потому что доступные уровни `modulus chain` для очередного `rescale` исчерпаны.

Bootstrapping-модуль

Bootstrapping-модуль выделен как отдельный компонент библиотеки. Он связан с анализом глубины и фиксирует конвейер:

$$\text{ModRaise} \rightarrow \text{CoeffToSlot} \rightarrow \text{EvalMod} \rightarrow \text{SlotToCoeff}.$$

Текущий `scalable target` — не плотная таблица преобразования, а `factorized FFT-like` реализация `CoeffToSlot` и `SlotToCoeff`. Плотное диагональное разложение остаётся `reference-path` для малых размеров.

Для `EvalMod` при нормализации

$$|u| \leq 2^{-10}$$

достаточен полином P_3 :

$$P_3(u) = u - 6.579736267393u^3.$$

Его ошибка аппроксимации на указанном интервале существенно меньше 10^{-9} ; более высокие степени P_5/P_7 увеличивают глубину и используются только при расширении интервала или более жёсткой цели.

Цель итогового преобразования:

$$\text{Bootstrap}(c) = c',$$

где должны выполняться условия:

$$\text{Dec}(c') \approx \text{Dec}(c),$$

$$\text{level}(c') > \text{level}(c).$$

В публичном отчёте `BootstrapReport` фиксируются:

- параметры входного `ciphertext`;
- параметры `ciphertext` на границе вычислительной глубины;

- число успешно выполненных умножений;
- следующий показатель степени, на котором вычисление останавливается;
- причина остановки;
- стадии конвейера и их статусы;
- состояние критериев $\text{Dec}(c') \approx \text{Dec}(c)$ и $\text{level}(c') > \text{level}(c)$.

Отчёт по криптостойкости профилей

Для CKKS-профиля вычисляется суммарный размер коэффициентного модуля:

$$\log_2(Q) \approx \sum_i \text{coeff_modulus_bits}_i.$$

Для каждого профиля библиотека сравнивает это значение с лимитами Microsoft SEAL:

$$\log_2(Q) \leq \text{CoeffModulus} :: \text{MaxBitCount}(N, \text{sec_level}).$$

В отчёте фиксируются проверки для уровней:

$$\text{tc128}, \quad \text{tc192}, \quad \text{tc256}.$$

Для текущих профилей:

Профиль	N	$\log_2(Q)$	tc128	tc192	tc256
basic_ckks	8192	200	PASS	FAIL	FAIL
depth_ckks	16384	280	PASS	PASS	FAIL

Общий уровень проекта определяется минимальным уровнем среди используемых профилей:

$$\min(\text{tc128}, \text{tc192}) = \text{tc128}.$$

Демонстрационные приложения

В проекте поддерживаются следующие запускные сценарии:

Приложение	Назначение
demo_secure_stats	Защищённое вычисление суммы и среднего.
demo_galois_key_optimization	Сравнение полного и ограниченного набора Galois-ключей.
demo_abft	Проверка ABFT-инвариантов для add, sub, mul и rotate.
demo_noise_growth	Анализ расхода глубины и роста ошибки.
demo_bootstrap_pipeline	Отчёт bootstrapping-модуля.
bench_ckks	Измерение времени, ошибки и размеров ciphertext/ключей.
bench_chain_accuracy	Калибровка влияния chain length, scale и рабочей битности на точность.
bench_bootstrap_parts	Измерение plaintext-умножения, линейного преобразования, суммы слотов и полинома.
bench_parallel_throughput	Измерение параллельной обработки независимых ciphertext.
demo_profile_report	Таблица используемых CKKS-профилей.
demo_security_report	Проверка профилей по лимитам Microsoft SEAL.